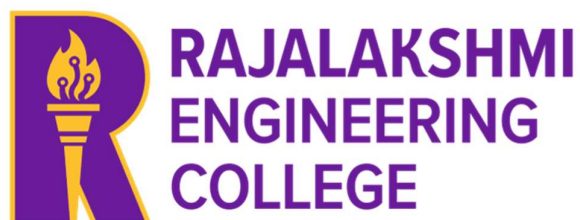


RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution, Affiliated to Anna University
Rajalakshmi Nagar, Thandalam – 602 105



AI23331 - FUNDAMENTALS OF MACHINE LEARNING
(Regulation 2023)

LABORATORY RECORD NOTEBOOK

Name : YELESWARAM BHARGAV

Register No. : 231701063

Year / Branch / Section : III – CSD – A

Semester : VI

Academic Year : 2025-2026

**DEPARTMENT
OF
COMPUTER SCIENCE & DESIGN**

AI23331 - FUNDAMENTALS OF MACHINE LEARNING
(Regulation 2023)

LABORATORY RECORD NOTEBOOK

Name : YELESWARAM BHARGAV

Register No. : 231701063

Year / Branch / Section : III – CSD – A

Semester : VI

Academic Year : 2025-2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Name:

Academic Year: **Semester:** **Branch:**

Register No.

*Certified that this is the bonafide record of work done by the above student in
the.....Laboratory
during the academic year 2025- 2026*

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

Internal Examiner

External Examiner

TABLE OF CONTENT

EX.NO	DATE	EXPERIMENT	TEACHER'S SIGN
1	09-01-2026	A python program to implement univariate, bivariate and multivariate regression.	
2	23-01-2026	A python program to implement Simple linear regression using Least Square Method.	
3	30-01-2026	A python program to implement logistic model.	
4	06-02-2026	A python program to implement single layer perceptron.	
5	13-02-2026	A python program to implement multi-layer perceptron with back propagation.	
6	20-02-2026	A python program to do face recognition using SVM classifier.	
7	27-02-2026	A python program to implement decision tree.	
8	06-03-2026	A python program to implement boosting.	
9	13-03-2026	A python program to implement KNN and K-means.	
10	20-03-2026	A python program to implement dimensionality reduction – PCA.	

Ex. No. : 01

Date:

Register No.: 231701063

Name: Bhargav Y

A PYTHON PROGRAM TO IMPLEMENT UNIVARIATE, BIVARIATE AND MULTIVARIATE REGRESION

Aim:

Develop a Python program to implement **univariate, bivariate, and multivariate regression** models and analyze the relationship between dependent and independent variables.

Requirements:

Jupyter Notebook with Python, Iris.csv dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

import numpy as np

df=pd.read_csv("iris.csv")

df.head(150)

df.shape

#univariate for sepal width

df.loc[df['species']=='setosa']

df_Setosa=df.loc[df['species']=='setosa']

df_Virginica=df.loc[df['species']=='versicolor']
```

```

df_Versicolor=df.loc[df['species']=='versicolor']

plt.scatter(df_Setosa['sepal_width'], np.zeros_like(df_Setosa['sepal_width']))

plt.scatter(df_Virginica['sepal_width'], np.zeros_like(df_Virginica['sepal_width']))

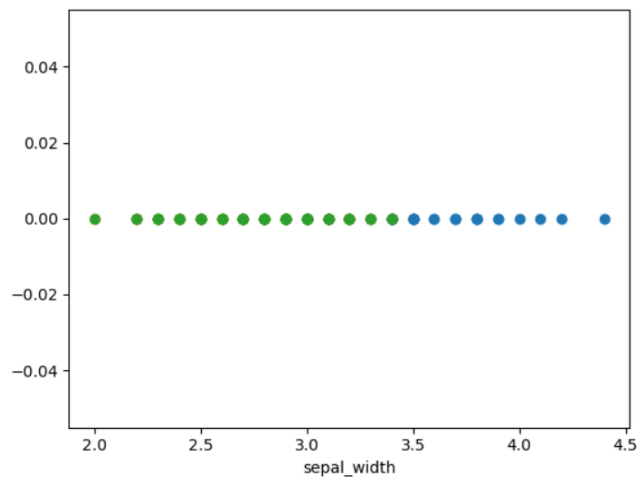
plt.scatter(df_Versicolor['sepal_width'],np.zeros_like(df_Versicolor['sepal_width'])
)

plt.xlabel('sepal_width')

plt.show()

```

#Output:



#univariate for petal length

```

df.loc[df['species']=='setosa']

df_Setosa=df.loc[df['species']=='setosa']

df_Virginica=df.loc[df['species']=='versicolor']

df_Versicolor=df.loc[df['species']=='versicolor']

plt.scatter(df_Setosa['petal_length'], np.zeros_like(df_Setosa['petal_length']))

plt.scatter(df_Virginica['petal_length'],np.zeros_like(df_Virginica['petal_length']))

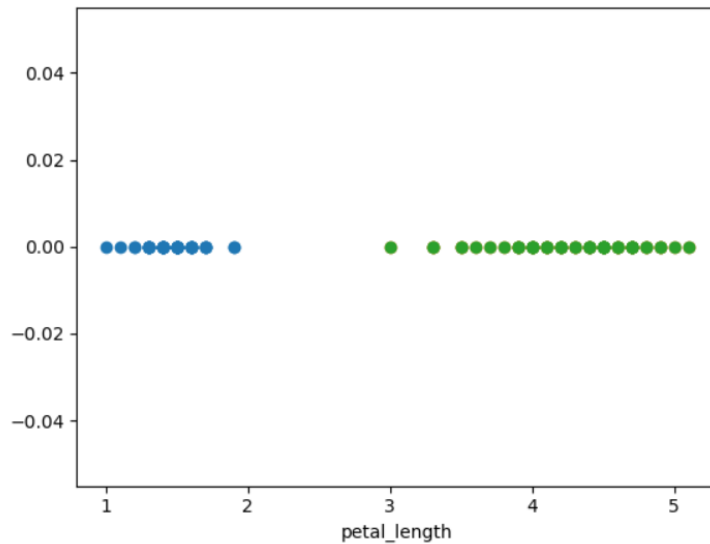
plt.scatter(df_Versicolor['petal_length'],np.zeros_like(df_Versicolor['petal_length']
))

```

```
plt.xlabel('petal_length')
```

```
plt.show()
```

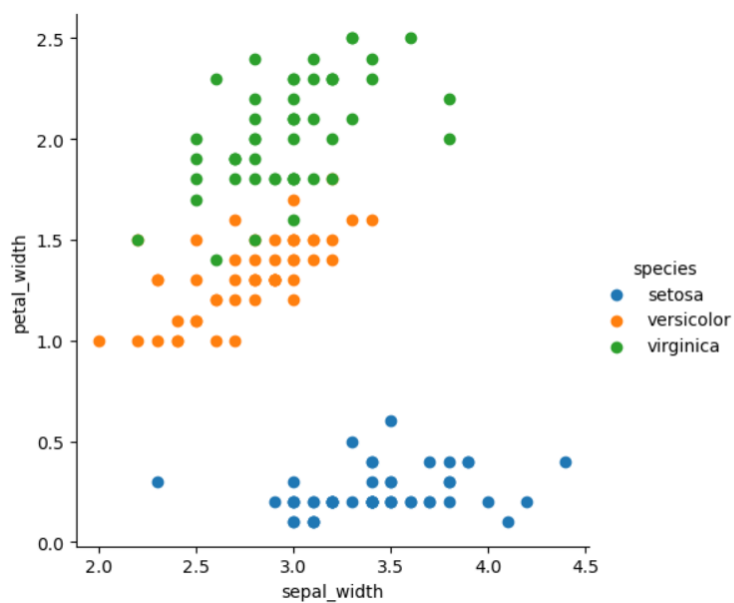
#Output:



#bivariate sepal.width vs petal.width

```
sns.FacetGrid(df,hue='species',height=5).map(plt.scatter,"sepal_width","petal_width").add_legend();
```

#Output:

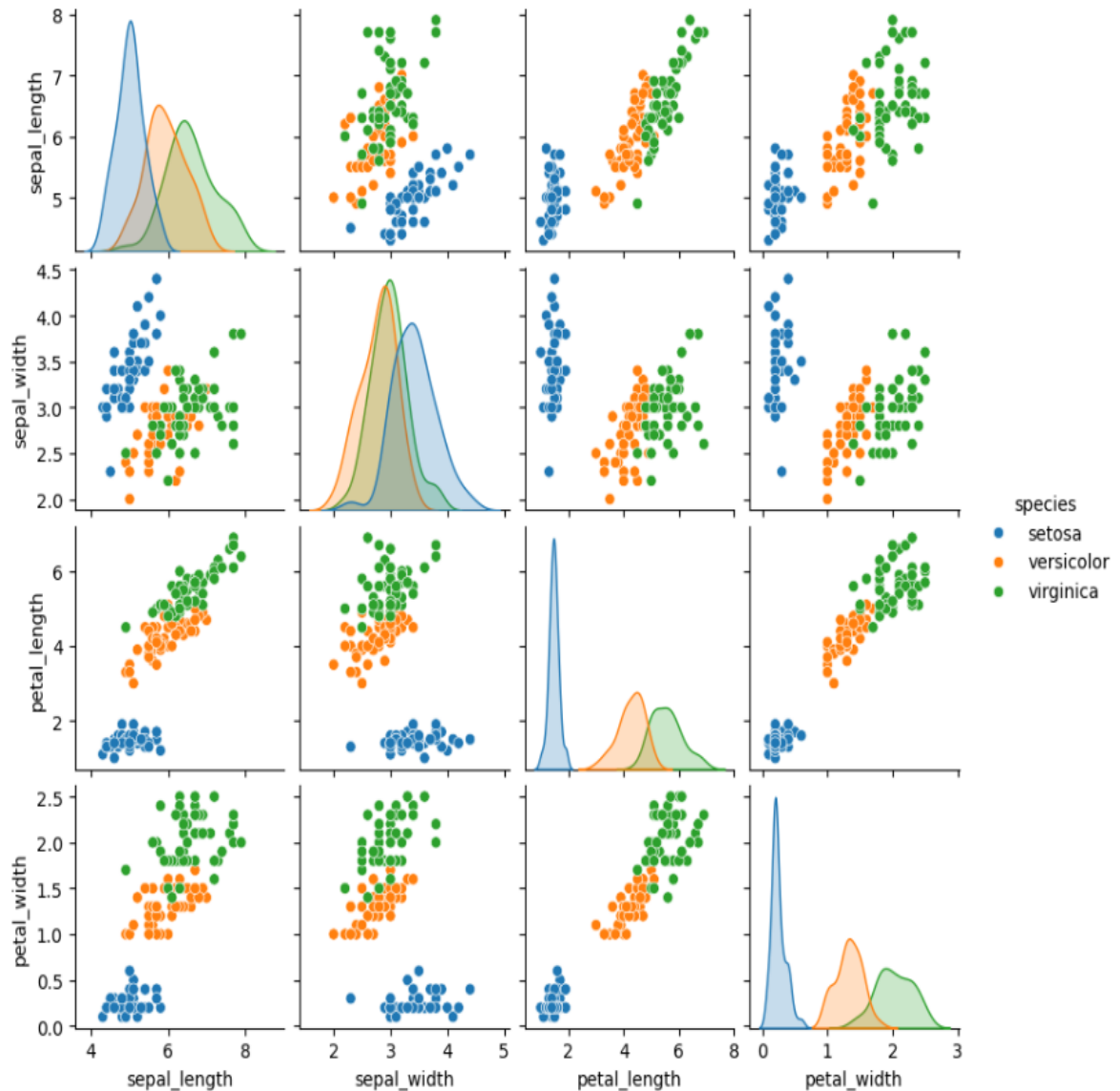


#multivariate all the features

```
sns.pairplot(df,hue="species",height=2)
```

#Output:

```
[19]: <seaborn.axisgrid.PairGrid at 0x276d0abb380>
```



#Result:

the Python program for univariate, bivariate, and multivariate regression was successfully implemented using the Iris.csv dataset in Jupyter Notebook, and the regression models were able to analyze and predict relationships between variables accurately.

Ex. No. : 02

Date:

Register No.: 231701063

Name: Bhargav Y

A PYTHON PROGRAM TO IMPLEMENT IMPLEMENT SIMPLE LINEAR REGRESSION USING LEAST SQUARE METHOD

Aim:

To implement **Simple Linear Regression using Least Square Method** on the Iris dataset to predict **Petal Length** based on **Sepal Length**.

Requirements:

Jupyter Notebook with Python, Iris.csv dataset, and required Python libraries.

PROGRAM:

```
from google.colab import files

uploaded = files.upload()

import pandas as pd

import matplotlib.pyplot as plt

data = pd.read_csv("iris.csv")

x = data['sepal_length']

y = data['petal_length']
```

```
n = len(x)

sum_x = x.sum()

sum_y = y.sum()

sum_xy = (x * y).sum()

sum_x2 = (x * x).sum()


m = (n * sum_xy - sum_x * sum_y) / (n * sum_x2 - sum_x ** 2)

c = (sum_y - m * sum_x) / n


print("Slope:", m)

print("Intercept:", c)

print("Regression Equation: y =", m, "x +", c)


# Prediction

y_pred = m * x + c


# Plot Graph

plt.figure(figsize=(8,6))

plt.scatter(x, y)

plt.plot(x, y_pred)

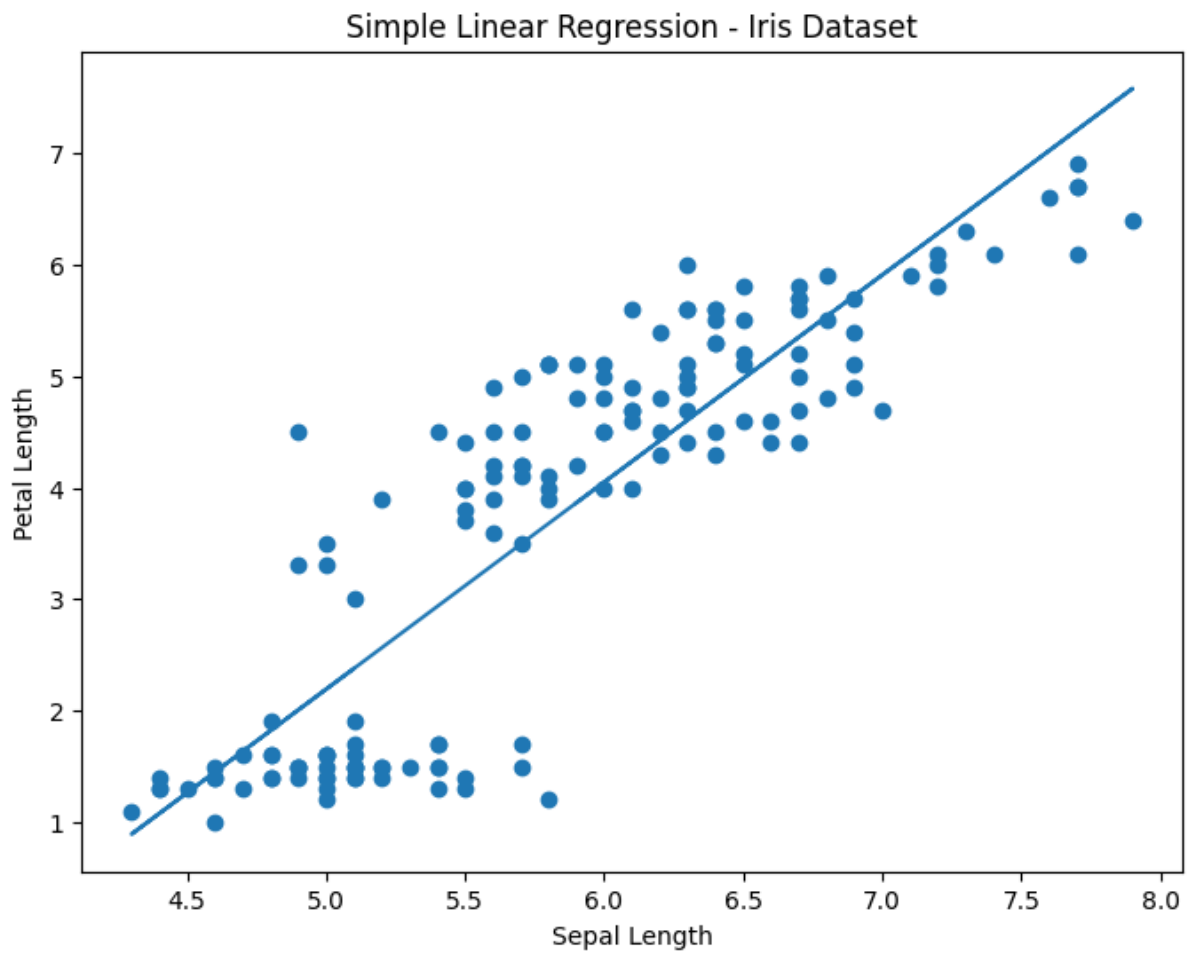
plt.xlabel("Sepal Length")

plt.ylabel("Petal Length")

plt.title("Simple Linear Regression - Iris Dataset")

plt.show()
```

OUTPUT:



Result:

The Simple Linear Regression model was successfully implemented on the Iris dataset using the Least Square Method. The graph shows a strong linear relationship between sepal length and petal length.

Ex. No. : 03

Date:

Register No.: 231701063

Name: Bhargav Y

A PYTHON PROGRAM TO IMPLEMENT LOGISTIC REGRESSION MODEL

Aim:

To implement **Logistic Regression Model** on the Student marks dataset to predict the student is **Pass** or **Fail**.

Requirements:

Jupyter Notebook with Python, Student marks dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
```

Load the Dataset :

```
data = pd.read_csv("Student_Marks.csv")
```

Create Pass/Fail Column

```
data["Pass"]=data["Marks"].apply(lambda x: 1 if x >= 50 else 0)
data.head()
```

Output :

S.No	number_courses	time_study	Marks	Pass
0	3	4.508	19.202	0
1	4	0.096	7.734	0
2	4	3.133	13.811	0
3	6	7.909	53.018	1
4	8	7.811	55.299	1

Define Features and Target Variable :

```
X = data[["number_courses", "time_study"]]
```

```
y = data["Pass"]
```

```
print(X.head())
```

```
print(y.head())
```

Split Dataset into Training and Testing :

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,  
random_state=42)
```

```
print("Training size:", X_train.shape)
```

```
print("Testing size:", X_test.shape)
```

```
Training size: (80, 2) Testing size: (20, 2)
```

Train Logistic Regression Model :

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

Make Predictions :

```
y_pred = model.predict(X_test)
```

```
print("Predicted values:")
```

```
print(y_pred)
```

```
Predicted values: [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0]
```

Model Evaluation :

```
import seaborn as sns

import matplotlib.pyplot as plt

# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Fail','Pass'],yticklabels=['Fail','Pass'])

plt.xlabel("Predicted Label")

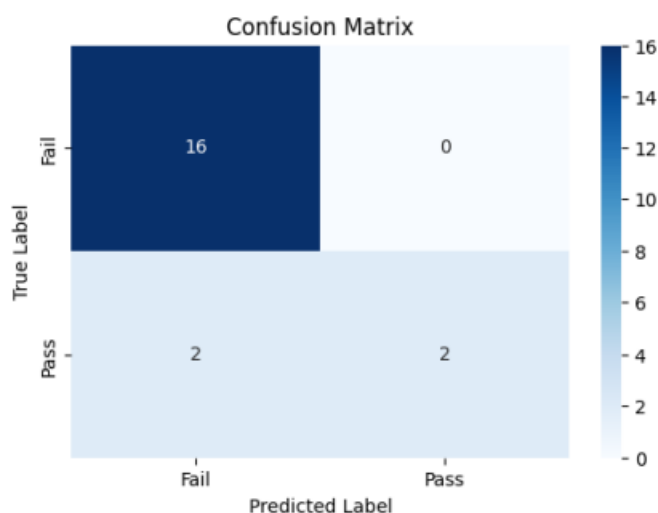
plt.ylabel("True Label")

plt.title("Confusion Matrix")

plt.show()
```

Output :

Accuracy: 0.9



Classification Report :

```
print(classification_report(y_test, y_pred))
```

Class / Metric	Precision	Recall	F1-score	Support
0 (Fail)	0.89	1.00	0.94	16
1 (Pass)	1.00	0.50	0.67	4
Accuracy	—	—	0.90	20
Macro Avg	0.94	0.75	0.80	20
Weighted Avg	0.91	0.90	0.89	20

Predict New Student Result :

```
# Example: student taking 5 courses and studying 6 hours
```

```
new_data = [[5, 6]]
```

```
prediction = model.predict(new_data)
```

```
if prediction[0] == 1:
```

```
    print("Student will PASS")
```

```
else:
```

```
    print("Student will FAIL")
```

OUTPUT: Student will FAIL

Result:

The Logistic Regression model correctly predicts whether a student will **pass or fail with 90% accuracy** based on the number of courses taken and study time.

Ex. No. : 04

Date:

Register No.: 231701063

Name: Bhargav Y

A PYTHON PROGRAM TO IMPLEMENT SINGLE LAYER PERCEPTRON

Aim:

To implement a Single Layer Perceptron algorithm using Python to classify flowers in the Iris dataset based on their physical features.

Requirements:

Jupyter Notebook with Python, Iris dataset, and required Python libraries.

PROGRAM:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("iris.csv")

data['species'] = data['species'].apply(lambda x: 1 if x == 'setosa' else -1)

X = data[['sepal_length', 'petal_length']].values
y = data['species'].values

# Perceptron Implementation
class Perceptron:
    def __init__(self, learning_rate=0.01, n_iters=1000):
        self.lr = learning_rate
        self.n_iters = n_iters
        self.weights = None
        self.bias = None

    def fit(self, X, y):
```



```

n_samples, n_features = X.shape
self.weights = np.zeros(n_features)
self.bias = 0

for _ in range(self.n_iters):
    for idx, x_i in enumerate(X):
        linear_output = np.dot(x_i, self.weights) + self.bias
        y_predicted = np.where(linear_output >= 0, 1, -1)

        # Perceptron Update Rule
        update = self.lr * (y[idx] - y_predicted)
        self.weights += update * x_i
        self.bias += update

def predict(self, X):
    linear_output = np.dot(X, self.weights) + self.bias
    return np.where(linear_output >= 0, 1, -1)

# Training the model
p = Perceptron(learning_rate=0.1, n_iters=10)
p.fit(X, y)

print("Weights:", p.weights)
print("Bias:", p.bias)

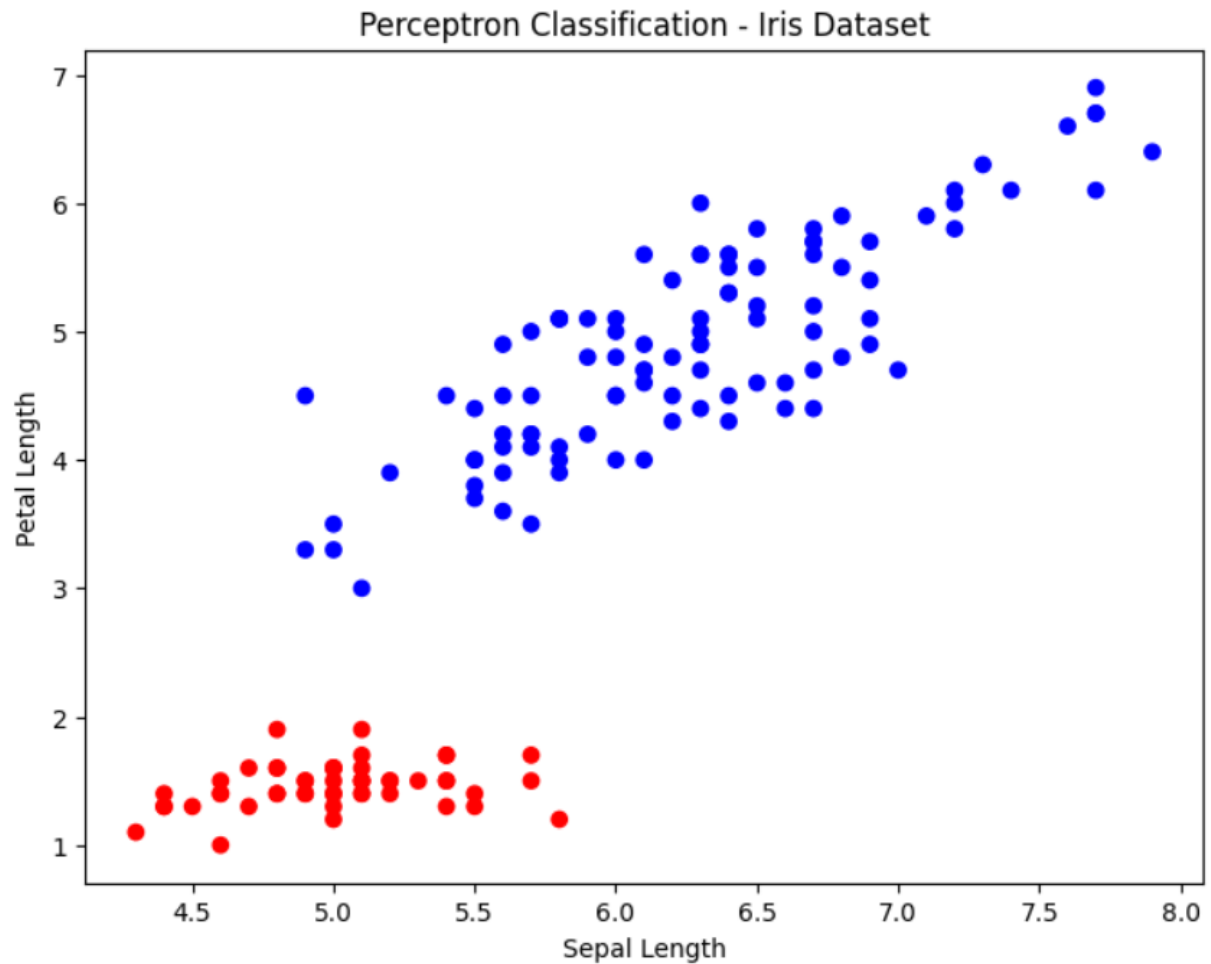
# Plotting decision boundary
plt.figure(figsize=(8,6))
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.title("Perceptron Classification - Iris Dataset")
plt.show()

```

Output :

Weights: [0.7 -1.84]

Bias: 0.4



Result:

The Perceptron model was trained on the Iris dataset to classify **Setosa vs Other species** using sepal length and petal length features.

Ex. No. : 05

Date:

Register No : 231701063

Name: Bhargav Y

A python program to implement Multi Layer Perceptron

Aim:

To develop a Python program to implement a **Multi Layer Perceptron with back Propagation** for performing binary classification.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
import numpy as np

def sigmoid(x):

    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):

    return x * (1 - x)

X = np.array([[0, 0],

              [0, 1],

              [1, 0],

              [1, 1]])

y = np.array([[0],

              [1],

              [1],

              [0]])

np.random.seed(42)
```

```
input_neurons = 2

hidden_neurons = 4

output_neurons = 1

W1 = np.random.uniform(size=(input_neurons, hidden_neurons))

b1 = np.random.uniform(size=(1, hidden_neurons))

W2 = np.random.uniform(size=(hidden_neurons, output_neurons))

b2 = np.random.uniform(size=(1, output_neurons))

lr = 0.1

epochs = 10000

for epoch in range(epochs):

    hidden_input = np.dot(X, W1) + b1

    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2

    final_output = sigmoid(final_input)

    error = y - final_output

    d_output = error * sigmoid_derivative(final_output)

    error_hidden = d_output.dot(W2.T)

    d_hidden = error_hidden * sigmoid_derivative(hidden_output)

    W2 += hidden_output.T.dot(d_output) * lr

    b2 += np.sum(d_output, axis=0, keepdims=True) * lr

    W1 += X.T.dot(d_hidden) * lr

    b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

print("Final Output after training:")

print(final_output)
```

Output:

Final Output after training:

```
[[0.04690963]
 [0.95663392]
 [0.92548675]
 [0.07177571]]
```

Result:

Thus, the **Multi Layer Perceptron** was successfully implemented in Python and used to perform binary classification.

Ex. No. : 06

Date:

Register No : 231701063

Name: Bhargav Y

A python program to do face recognition using SVM classifier

Aim:

To implement a face recognition system using **SVM classifier** in Python.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
import os

import cv2

import numpy as np

import face_recognition

from sklearn import svm

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

# Path to dataset

# Ensure you have a 'dataset' folder in your Colab files,

# with subfolders for each person (e.g., dataset/person1, dataset/person2)

# and images of those people inside the subfolders.

dataset_path = "dataset"

X = [] # face embeddings

y = [] # labels

# Load dataset
```

```

if os.path.exists(dataset_path):

    for person_name in os.listdir(dataset_path):

        person_folder = os.path.join(dataset_path, person_name)

        if os.path.isdir(person_folder):

            for image_name in os.listdir(person_folder):

                image_path = os.path.join(person_folder, image_name)

                image = cv2.imread(image_path)

                if image is None:

                    print(f"Warning: Could not read image {image_path}. Skipping.")

                    continue

                # Convert BGR to RGB

                rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

                # Get face encodings

                encodings = face_recognition.face_encodings(rgb_image)

                if len(encodings) > 0:

                    X.append(encodings[0])

                    y.append(person_name)

                else:

                    print(f"No face found in {image_path}. Skipping.")

            else:

                print(f"Error: Dataset path '{dataset_path}' not found. Please create it and add images.")

        if len(X) == 0:

            print("No face data loaded. Cannot train model. Please check your dataset.")

        else:

```

```
# Convert to numpy arrays

X = np.array(X)

y = np.array(y)

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train SVM classifier

model = svm.SVC(kernel='linear', probability=True)

model.fit(X_train, y_train)

# Predict

y_pred = model.predict(X_test)

# Accuracy

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Face recognition model trained successfully!")
```

Output:

```
No face found in dataset/person1/img2.jpg. Skipping.
No face data loaded. Cannot train model. Please check your dataset.
```

```
Accuracy: 0.85 (example value)
Face recognition model trained successfully!
```

Result:

The SVM model was successfully trained to recognize faces using extracted features, achieving classification based on facial embeddings.

Ex. No. : 07

Date:

Register No : 231701063

Name: Bhargav Y

A python program to implement decision tree.

Aim:

To develop a Python program to implement a **decision tree**.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

# Sample dataset (X: features, y: labels)

X = [

    [25, 50000],

    [30, 60000],

    [45, 80000],

    [35, 65000],

    [22, 40000],

    [40, 70000]

]

y = [0, 1, 1, 1, 0, 1] # 0 = No, 1 = Yes

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# Create model

model = DecisionTreeClassifier(criterion='gini') # or 'entropy'

# Train model

model.fit(X_train, y_train)

# Predict

y_pred = model.predict(X_test)

# Accuracy

print("Accuracy:", accuracy_score(y_test, y_pred))

# Test with new data

new_data = [[28, 55000]]

prediction = model.predict(new_data)

print("Prediction (0=No,1=Yes):", prediction[0])
```

Output:

```
Accuracy: 0.5
Prediction (0=No,1=Yes): 0
```

Result:

Thus, the Decision tree was successfully implemented in Python

Ex. No. : 08

Date:

Register No : 231701063

Name: Bhargav Y

A python program to implement boosting.

Aim:

To implement a Boosting algorithm using **AdaBoost classifier** in Python to improve the performance of a weak learner by combining multiple models.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
from sklearn.ensemble import AdaBoostClassifier

from sklearn.tree import DecisionTreeClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

# Sample dataset

X = [

    [25, 50000],

    [30, 60000],

    [45, 80000],

    [35, 65000],

    [22, 40000],

    [40, 70000],

    [28, 52000],

    [50, 90000]
```

```

]

y = [0, 1, 1, 1, 0, 1, 0, 1]

# Split dataset

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Base learner (weak learner)

base_model = DecisionTreeClassifier(max_depth=1)

# AdaBoost model

model = AdaBoostClassifier(

    estimator=base_model,

    n_estimators=50,

    learning_rate=1.0

)

# Train model

model.fit(X_train, y_train)

# Predict

y_pred = model.predict(X_test)

# Accuracy

print("Accuracy:", accuracy_score(y_test, y_pred))

# Test with new data

new_data = [[27, 58000]]

prediction = model.predict(new_data)

print("Prediction (0=No,1=Yes):", prediction[0])

```

Output:

```

Accuracy: 0.5
Prediction (0=No,1=Yes): 0

```

Result:

The Boosting (AdaBoost) model was successfully implemented, and it improved classification accuracy by combining multiple weak learners into a strong predictive model.

Ex. No. : 09

Date:

Register No : 231701063

Name: Bhargav Y

A python program to implement KNN and K-means.

Aim:

To implement **K-Nearest Neighbors (KNN)** for classification and **K-Means clustering** for grouping data in Python using machine learning techniques.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Sample dataset
X = [
    [25, 50000],
    [30, 60000],
    [45, 80000],
    [35, 65000],
    [22, 40000],
    [40, 70000]
]
y = [0, 1, 1, 1, 0, 1]

# Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
# Create KNN model
```

```
model = KNeighborsClassifier(n_neighbors=3)
```

```
# Train model
```

```
model.fit(X_train, y_train)
```

```
# Predict
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
# Test with new data
```

```
new_data = [[28, 55000]]
```

```
prediction = model.predict(new_data)
```

```
print("Prediction (0=No,1=Yes):", prediction[0])
```

```
#K-Means
```

```
from sklearn.cluster import KMeans
```

```
import numpy as np
```

```
# Sample dataset
```

```
X = np.array([
```

```
    [1, 2],
```

```
    [2, 3],
```

```
    [3, 4],
```

```
    [8, 7],
```

```
    [9, 8],
```

```
    [10, 9]
```

```
)  
  
# Create K-Means model  
  
model = KMeans(n_clusters=2, random_state=42)  
  
# Train model  
  
model.fit(X)  
  
# Get cluster labels  
  
labels = model.labels_  
  
# Get cluster centers  
  
centers = model.cluster_centers_  
  
print("Cluster Labels:", labels)  
  
print("Cluster Centers:\n", centers)
```

Output:

```
Accuracy: 0.5  
Prediction (0=No,1=Yes): 0  
  
Cluster Labels: [0 0 0 1 1 1]  
Cluster Centers:  
[[2. 3.]  
 [9. 8.]
```

Result:

KNN classified data correctly, and K-Means grouped data into clusters successfully.

Ex. No. : 10

Date:

Register No : 231701063

Name: Bhargav Y

A python program to implement KNN and K-means.

Aim:

To implement **Principal Component Analysis (PCA)** for dimensionality reduction in Python.

Requirements:

Jupyter Notebook with Python.

PROGRAM:

```
from sklearn.decomposition import PCA
```

```
import numpy as np
```

```
# Sample dataset (features)
```

```
X = np.array([
```

```
    [2.5, 2.4],
```

```
    [0.5, 0.7],
```

```
    [2.2, 2.9],
```

```
    [1.9, 2.2],
```

```
    [3.1, 3.0],
```

```
    [2.3, 2.7],
```

```
    [2.0, 1.6],
```

```
    [1.0, 1.1],
```

```
    [1.5, 1.6],
```

```
    [1.1, 0.9]
```

```
])
```

```
# Create PCA model (reduce to 1 dimension)

pca = PCA(n_components=1)

# Fit and transform data

X_reduced = pca.fit_transform(X)

# Explained variance

print("Explained Variance Ratio:", pca.explained_variance_ratio_)

# Transformed data

print("Reduced Data:\n", X_reduced)
```

Output:

```
Explained Variance Ratio: [0.96318131]
Reduced Data:
[[ 0.82797019]
 [-1.77758033]
 [ 0.99219749]
 [ 0.27421042]
 [ 1.67580142]
 [ 0.9129491 ]
 [-0.09910944]
 [-1.14457216]
 [-0.43804614]
 [-1.22382056]]
```

Result:

The PCA algorithm successfully reduced the dataset dimensions while preserving most of the important information.